AI AGENT DEVELOPMENT

Coding Assessment

⏱ Duration: 60 Minutes | 📁 Dataset: 5,000 Transactions | 👥 10 Users

1. Assessment Overview

This assessment evaluates your ability to design and implement a tool-augmented LLM agent that reasons over structured financial data. You are not expected to build a production system — the focus is on clean thinking, correct tool design, and appropriate use of language models.

What We're Looking For

Can you decide what the LLM should handle vs. what code should handle? Can your agent produce accurate answers to natural language questions without feeding thousands of rows into the model?

Aspect	Details
Duration	60 minutes
Environment	Your laptop — internet, IDE, and AI coding assistants permitted
Language	Python — any standard libraries allowed
LLM Access	Use your own LLM Key
Submission	Runnable python file that creates conversational agent

2. The Problem: Multi-User Finance Agent

You are given a CSV file of 5,000 synthetic bank transactions covering 10 fictional users across Q1 2025 (January–March). Your task is to build a conversational agent that answers natural language questions about individual users' spending, and compares spending across users.

2.1 Dataset

File: synthetic_transactions_5k.csv

Download csv from https://github.com/hankai777/spend-dataset

Format:

Column	Type	Description	Example
date	string	Transaction date (YYYY-MM-DD)	2025-01-15
name	string	User name (one of 10 fixed users)	Alice
item	string	Description of purchase	Grocery Store
amount	float	Transaction amount in USD	47.82
category	string	Spending category (6 values)	Food

Users: Alice, Bob, Charlie, David, Eve, Frank, Grace, Heidi, Ivan, Judy

Categories: Food, Electronics, Entertainment, Transport, Housing, Health

Date range: 2025-01-01 to 2025-03-31 | Amount range: $5.09 to $2,498.56

3. Required Tools to Implement

Your agent must use at least two Python functions as tools. These functions filter and aggregate data using Pandas before anything is passed to the LLM. The LLM must never receive raw rows.

Tool 1: get_user_transactions(name, category=None, month=None)

Returns a filtered summary of transactions for a given user. Optionally filter by spending category and/or month number (1=January, 2=February, 3=March).

Returns a dict or DataFrame summary — not raw rows. Example output:

```
# Example return for get_user_transactions("Alice", category="Food", month=2)
# Returns: {
#   "user": "Alice",
#   "category": "Food",
#   "month": "February",
#   "transaction_count": 12,
#   "total_spent": 384.20,
#   "avg_per_transaction": 32.02,
```

```
#   "sample_items": ["Grocery Store", "Cafe", "Takeout"]
# }
```

Tool 2: calculate_user_metrics(name, category=None, metric="total")

Returns computed metrics for a user, optionally filtered by category. The metric parameter accepts: "total", "average", "max", "min", or "count".

Returns a single-value dict. Example:

```
# Example: calculate_user_metrics("Heidi", metric="average")
# Returns: { "user": "Heidi", "metric": "average", "value": 214.57, "currency": "USD" }
```

> ⚠️ **Critical Constraint**
>
> Never pass raw CSV rows to the LLM. Your tools must do the heavy lifting — aggregation, filtering, and summarisation happen in Python. The LLM receives only the structured result. This is what separates a good agent from a naive one.

4. Required Test Queries

Your agent must correctly answer all five queries below. These will be run verbatim during the evaluation.

#	Query	What It Tests
Q1	How much did Alice spend on Food in February?	Single user + category + month filter
Q2	Who spent more on Electronics: Bob or Charlie?	Cross-user comparison + category filter
Q3	What is the average transaction amount for Heidi?	Aggregation (average) for single user
Q4	Which category did David spend the most on overall?	Grouping + ranking within a user
Q5	List the top 3 spenders across all users.	Full-dataset ranking, multi-user aggregation

Handling Ambiguity

Your agent will be tested on ambiguous queries. Think about how to handle these gracefully:

- 'Show me the biggest spender' — no category or time window specified. Should the agent ask for clarification, or make a reasonable default assumption and state it?
- 'How did Alice do in Q1?' — vague metric. What does 'do' mean? Spend the most? Have the most transactions?
- 'Compare Bob and Charlie' — no category. Does the agent compare total spend? All categories?

Documenting your ambiguity-handling strategy (even in comments) will be evaluated.

7. Stretch Goals ✦

If you complete the core requirements early, attempt one or more of the following. These are optional but demonstrate senior-level thinking. Clearly mark which stretch goals you attempted.

Stretch A - The Visualizer

Add a tool that can generate and save a Matplotlib/Seaborn chart (e.g., a bar chart comparing users) based on the user's request. For Example

- Create a bar chart showing Alice's spending breakdown by category for the whole quarter.
- Show me a visual comparison of Bob vs. Charlie's total monthly spending

Stretch B — Multi-Turn Conversation Memory

Enable the agent to maintain context across multiple turns without re-reading the CSV on every call. For example:

- User: "How much did Alice spend on Food?"
- Agent: "$842.50 across Q1."
- User: "What about in February only?" — agent correctly interprets 'What about' as referring to Alice and Food.

Implement a simple conversation history that carries user/category context forward across turns.

Stretch C — Comparative / Ranking Queries

Handle open-ended ranking questions that the agent must decompose across all 10 users:

- 'Who is the biggest spender overall?'
- 'Rank all users by Food spending.'
- 'Which user had the most transactions in March?'

This requires either iterating over all users in the tool, or designing a new get_all_users_summary() tool. Consider the efficiency implications of each approach.

8. Submission Checklist

1. Single .py file or .ipynb notebook that runs end-to-end
2. Both required tools implemented (get_user_transactions, calculate_user_metrics)
3. All 5 test queries answered correctly with printed output
4. No raw CSV rows passed to the LLM at any point
5. Brief README comment at the top explaining your design decisions
6. (Optional) Clearly labelled stretch goal sections

Good Luck

We're evaluating how you think about the boundary between LLMs and code — not just whether the code runs. Clear reasoning, tidy tool design, and honest handling of edge cases matter as much as correctness.